JAVA MINI PROJECT

BLOOD BANK INVENTORY AND DONOR MATCHER

A simple system to manage blood stock and identify compatible donors.

PRESENTED BY

M. Shiva Rama Krishna • 2620030344

Ganesh • 26200371



BLACK × YELLOW

Project Abstract

THE NEED

Blood availability must be checked quickly during emergencies. Manual records can be slow and difficult to maintain.

THE SOLUTION

A Java-based application stores blood stock and donor details, then searches for a suitable blood group.

THE OUTCOME

Clear records, faster searching, and a simple way to view available units and possible donor matches.

Problem Statement

- Paper-based records can become outdated or duplicated.
- Searching donor details manually takes time.
- Blood group and available-unit information must be easy to check.
- A simple system can organize records and reduce avoidable mistakes.

Goal: make blood inventory and donor lookup simple, organized, and fast.

Project Objectives

01 Manage stock

Store blood group and unit count.

02 Register donors

Keep donor name, group, and contact.

03 Find matches

Search by required blood group.

04 Validate input

Reject invalid units or missing values.

05 Save records

Use files for basic data persistence.

06 Apply Java

Demonstrate core course outcomes.

System Workflow

01
Start
Open the application

→

02
Choose action
Stock / donor / search

→

03
Enter details
Validate user input

→

04
Process
Update or search records

→

05
Show result
Display match or stock

Input → Validation → Processing → Output

Key Features

Blood stock

View and update available units.

Donor records

Register and display donor details.

Group search

Find donors with the requested group.

Input checks

Check values before saving.

File storage

Save and load basic records.

Simple output

Show clear messages to the user.

System Design

USER INTERFACE

Menu
Input prompts
Result messages

BUSINESS LOGIC

Blood group search
Stock update
Validation

DATA STORAGE

Donor records
Blood unit counts
Text files

User Interface → Java Logic → Stored Records

CO1 — Data Types

Concept

Java data types define what kind of value a variable can store.

JAVA EXAMPLE

```
String donorName = "Ravi";
String bloodGroup = "O+";
int availableUnits = 12;
boolean eligible = true;
System.out.println(bloodGroup + ": " + availableUnits);
```

Project use: names and groups use String, unit counts use int, and yes/no flags use boolean.

CO2 — If-Else, Switch & Loops

JAVA EXAMPLE

```
if (units > 0) {
    System.out.println("Stock available");
} else {
    System.out.println("Out of stock");
}

switch (choice) {
    case 1: System.out.println("View stock"); break;
    case 2: System.out.println("Search donor"); break;
    default: System.out.println("Invalid choice");
}
```

If-else makes decisions • switch handles menu choices • loops repeat menu or search tasks.

CO3 — Methods & Arrays

JAVA EXAMPLE

```
static int totalUnits(int[] stock) {
    int total = 0;
    for (int units : stock) {
        total += units;
    }
    return total;
}

int[] stock = {8, 12, 5, 4};
System.out.println(totalUnits(stock));
```

Methods organize reusable logic. A 1D array can hold unit counts; a 2D array can represent group-by-location stock.

CO4 — OOP & Exceptions

CLASS

Donor describes the data and actions for a donor.

OBJECT

Each donor record is an object created from Donor.

EXCEPTION

try-catch can handle invalid number input.

JAVA EXAMPLE

```
try {
    int units = Integer.parseInt(input);
    if (units < 0) throw new IllegalArgumentException();
} catch (NumberFormatException e) {
    System.out.println("Enter a valid number.");
} catch (IllegalArgumentException e) {
    System.out.println("Units cannot be negative.");
}
```

CO5 — Strings, Files & Libraries

JAVA EXAMPLE

```
String group = "O+";
if (group.equalsIgnoreCase(requiredGroup)) {
    System.out.println("Possible group match");
}

// Example file-writing idea
try (java.io.FileWriter writer =
         new java.io.FileWriter("donors.txt", true)) {
    writer.write("Ravi,O+\n");
} catch (java.io.IOException e) {
    System.out.println("Could not save record.");
}
```

Strings compare names and blood groups. File handling saves records. Java libraries provide ready-made classes.

CO6 — Collections & Streams

JAVA EXAMPLE

```
import java.util.ArrayList;
import java.util.List;

List<String> donors = new ArrayList<>();
donors.add("Ravi - O+");
donors.add("Maya - A+");
donors.add("Arun - O+");

donors.stream()
      .filter(d -> d.endsWith("O+"))
      .forEach(System.out::println);
```

Collections store a flexible number of donor records. Streams filter and process matching entries.

Donor Matching Logic

- Read the required blood group from the user.
- Check each donor record in the stored list.
- Compare the donor group with the requested group.
- Display records that meet the matching rule.
- In a real blood bank, compatibility must be verified by qualified medical staff.

Educational prototype only — not a substitute for clinical compatibility testing.

Benefits & Future Scope

BENEFITS

- Faster record lookup
- Organized stock information
- Basic input validation
- Clear menu-driven workflow

FUTURE SCOPE

- Database integration
- Secure staff login
- Donor eligibility reminders
- Search by location and availability

CONCLUSION

A SIMPLE IDEA.
A USEFUL IMPACT.



The project demonstrates how Java can organize blood stock, store donor details, and help users search records. It applies the six course outcomes in one practical example.

THANK YOU

Questions?